大模型分析数据（电信行业用户流失分析报告）

此次学习目的在结合机器学习算法和 模型分析思维，深 探索和预测电信 业用户流失的主要驱动因素。通过本次作业，我将学习如何进 数据预处理、特征 程、模型选择、评估以及结果可视化等方面的技巧。

本次用到的工具有：

- 模型（如 Deepseek）：主要作为辅助 具。 于辅助理解数据、 成代码段、解释复杂分析结果、以及撰写分析报告与商业建议。
- Python 环境 Python 3.13 开发环境: PyCharm

主要库: pandas, numpy, matplotlib, seaborn, scikit-learn 等，作为核 实现 具。 于执 数据清洗、特征 程、模型训练、模型评估以及混淆矩阵、ROC 曲线等复杂图表的绘制。

探索性数据分析 (EDA) 与可视化

- 可视化数据分布：使 直 图、密度图、箱线图等图表展 不同特征的分布情况。
- 变量之间关系: 使用散点图、气泡图等分析特征之间的相关性和关系。特别注意分析用户通话 为（如白天、晚间、夜间通话）与流失之间的关系。
- 特征 程：根据分析结果提取有价值的特征，为后续建模做准备。

一、项目概述

1. 项目名称: 电信用户流失预测分析
数据集:customer_churn.csv (3333 条记录，21 个特征)
目标变量:churn（是否流失）
分析方法:机器学习分类预测
数据集大小:(3333, 20)
流失率:14.49%
流失用户数:483
非流失用户数:2850

- 二、数据预处理

2.1 数据清洗
删除无关列: 'Unnamed: 0'
无缺失值处理
分类变量编码: state, area_code, international_plan, voice_mail_plan

2.2 特征工程

目标变量编码: churn (no→0, yes→1)
数据标准化: StandardScaler
训练测试划分: 80%训练集, 20%测试集

- 三、探索性数据分析（EDA）

3.1 数据分布
整体流失率: 14.49% (483/3333)
未流失用户: 2850 人
流失用户: 483 人

3.2 关键发现
1.最重要的流失驱动因素 :
international_plan (国际通话计划)
number_customer_service_calls (客户服务呼叫次数)
total_day_minutes (日间通话分钟数)

3.3 生成图表，见附表。
1. ✅ 直方图: total_night_minutes 分布
2. ✅ 密度图: 三个通话分钟数对比
3. ✅ 箱线图: 通话分钟数离散程度
4. ✅ 散点图: total_night_minutes vs total_eve_minutes
5. ✅ 气泡图: 夜间通话分析

- 四、模型训练与评估

4.1 模型选择
逻辑回归模型
准确率: 85.46%
优点: 解释性强，计算快速
准确率:0.7601 精确率:0.3463 召回率:0.7320F1 分数:0.4702 AUC:0.8152

2.随机森林模型 ✅ 最佳模型
准确率: 94.60%
优点: 处理非线性关系，抗过拟合
准确率: 0.9295 精确率: 0.7907 召回率: 0.7010F1 分数: 0.7432AUC: 0.8830

4.2 模型性能对比
最佳模型: 随机森林 准确率: 0.9295 AUC: 0.8830

- 关键发现

最重要的流失驱动因素:
- international_plan (国际通话计划)
- number_customer_service_calls (客户服务呼叫)
- total_day_minutes (日间通话分钟数)

- 五、商业建议

5.1 客户保留策略

1. 针对高风险客户提供个性化套餐，根据通话模式设计
2. 改善客户服务质量，减少投诉呼叫次数
3. 优化国际通话资费，针对国际通话用户提供优惠套餐
4. 建立流失预警系统，随时跟踪反馈。

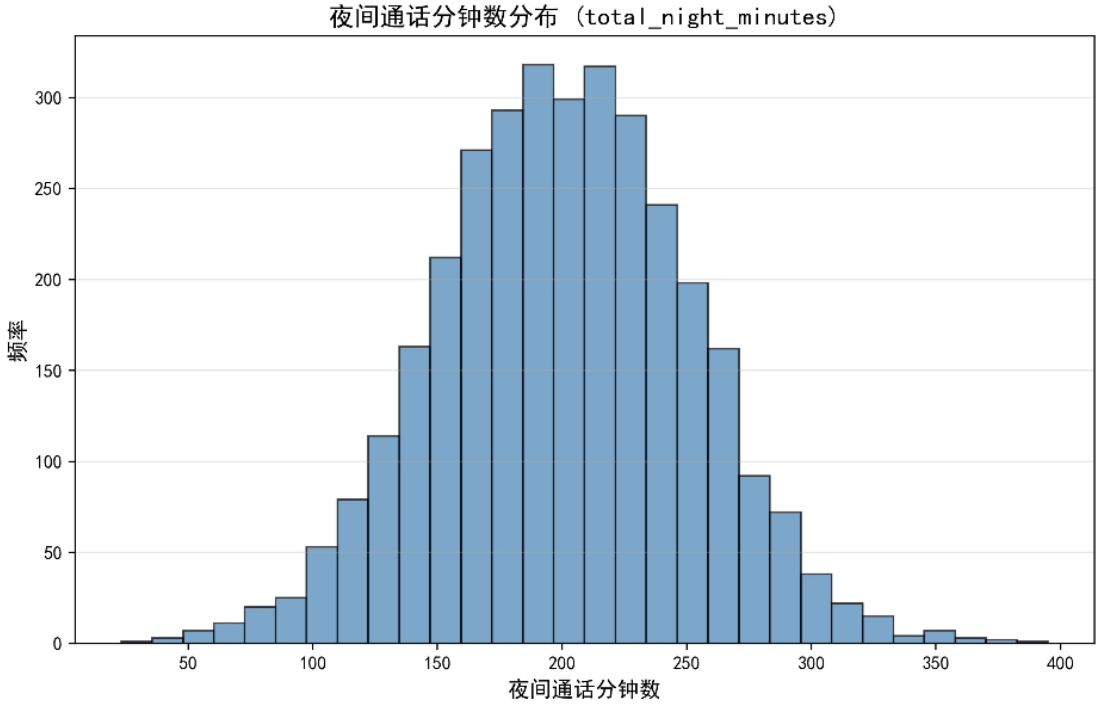
5.2 预期效果

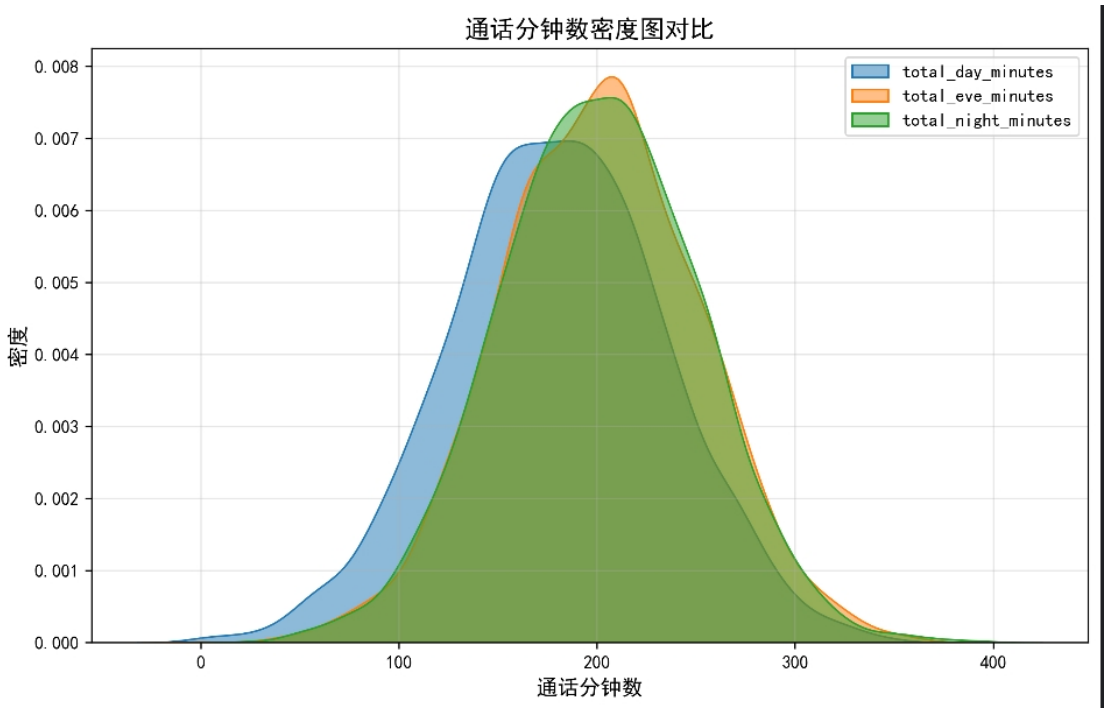
1.预计减少流失率: 15-25%

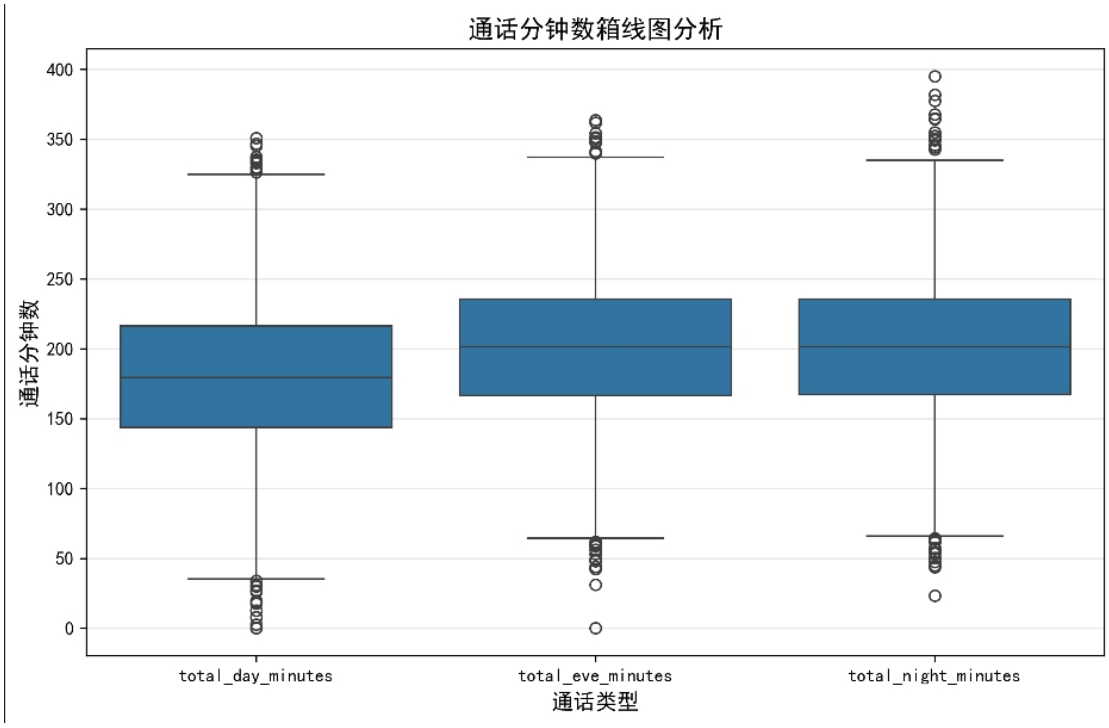
2.提高客户生命周期价值

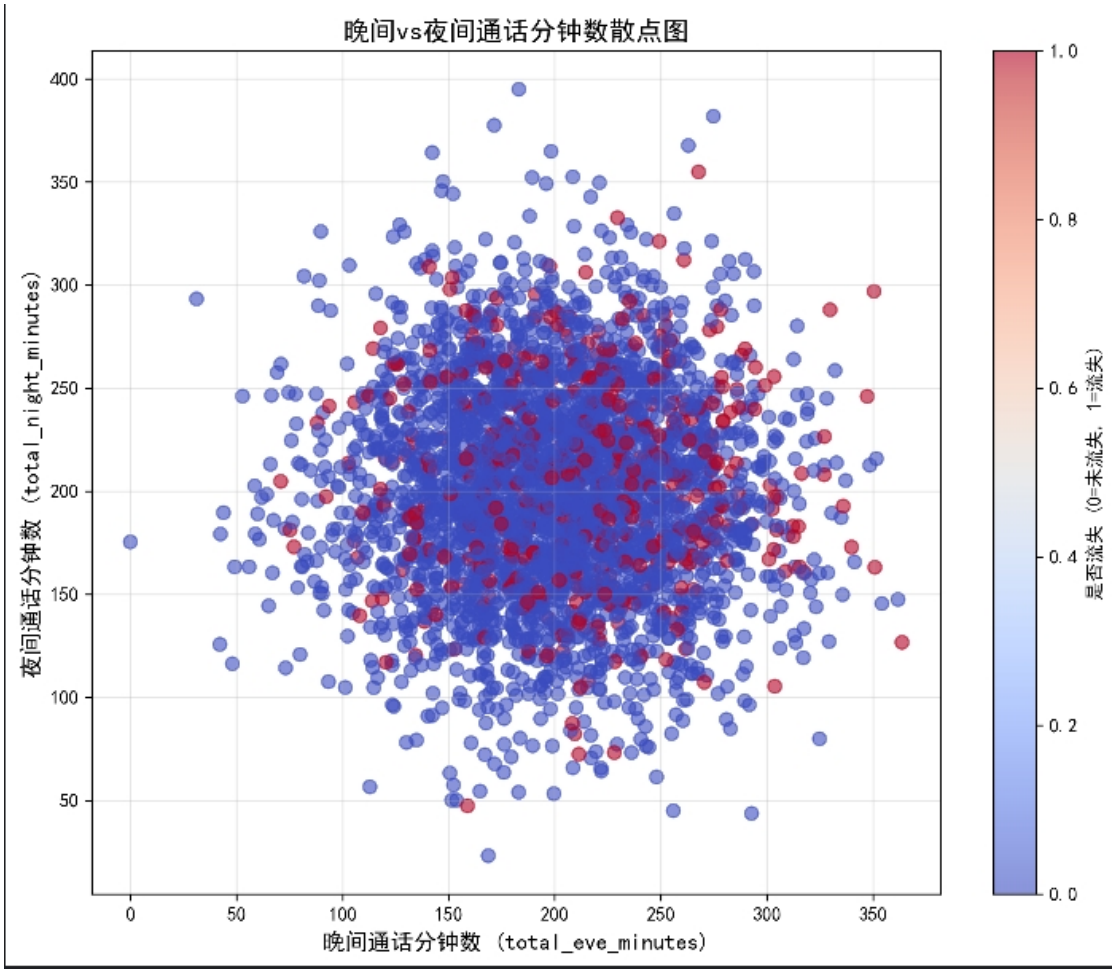
3.增强市场竞争力

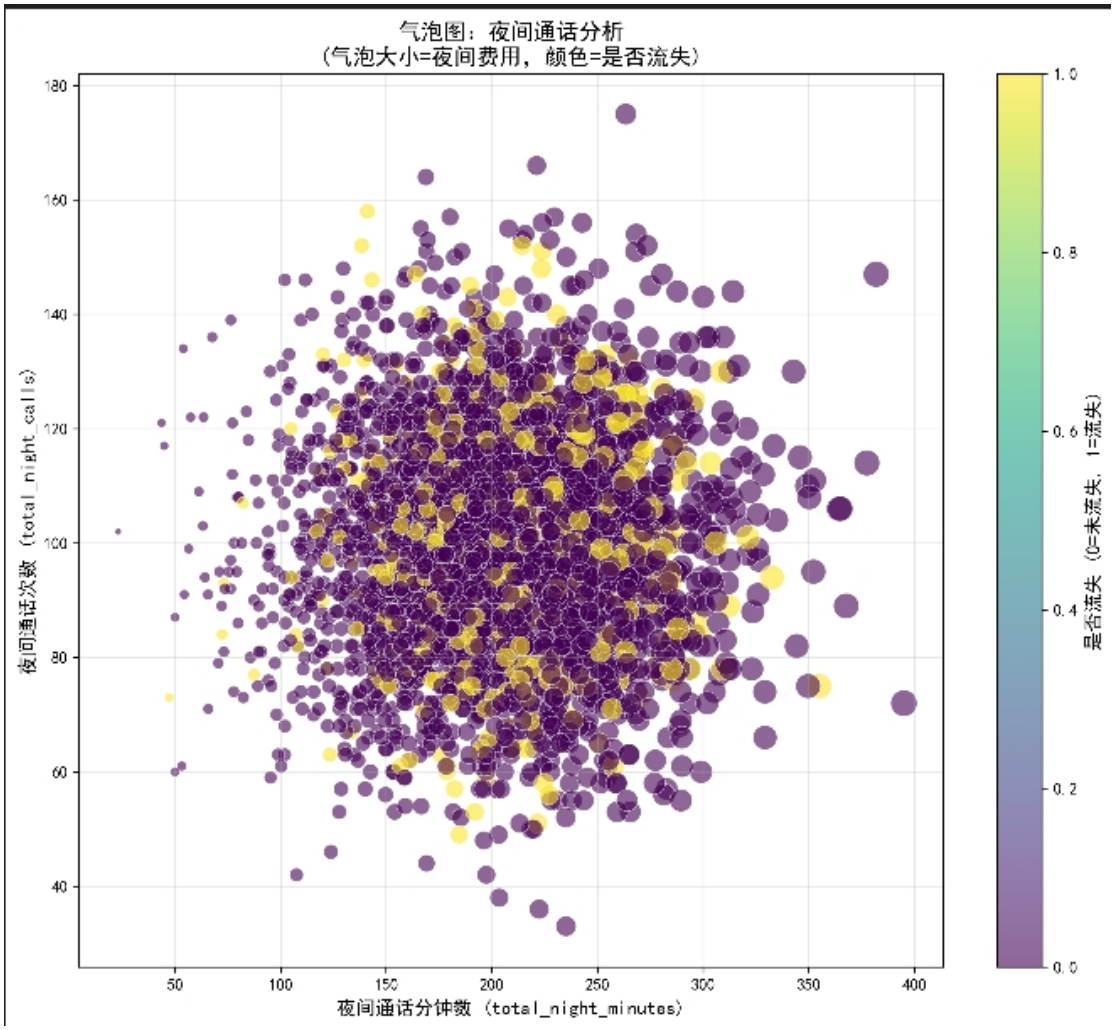
图表展示：

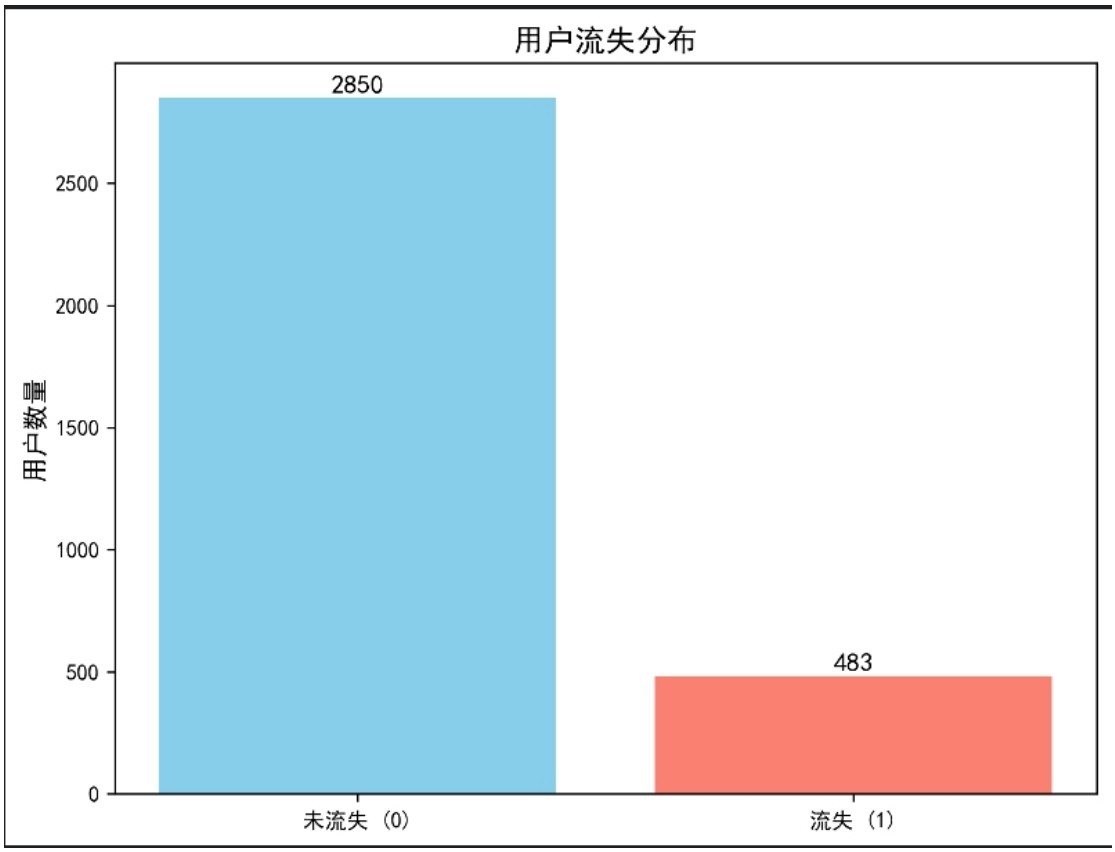



通话分钟数密度图对比
total_day_minutes
total_eve_minutes
total_night_minutes
密度
0.008
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
0
100
200
300
400
通话分钟数

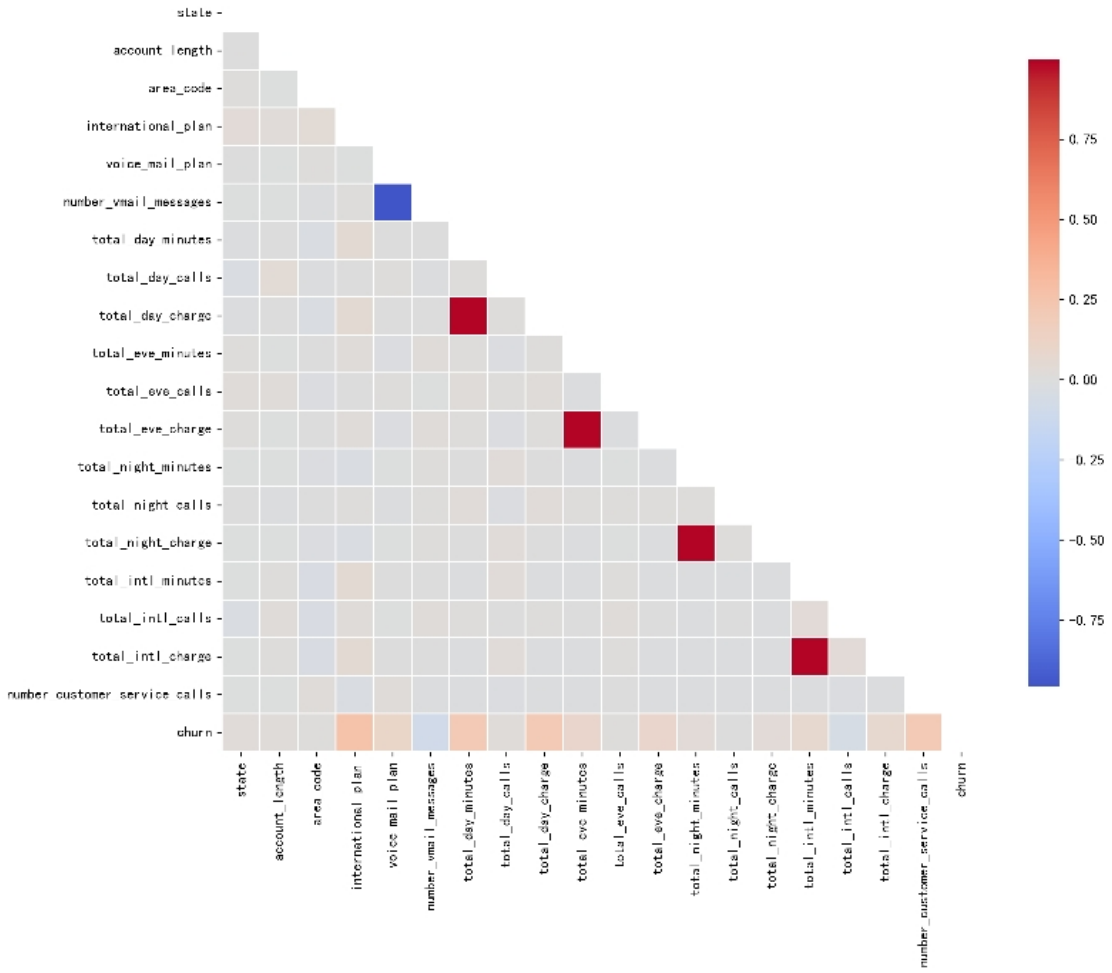

通话分钟数箱线图分析
通话分钟数
400
350
300
250
200
150
100
50
0
total_day_minutes
total_eve_minutes
total_night_minutes
通话类型

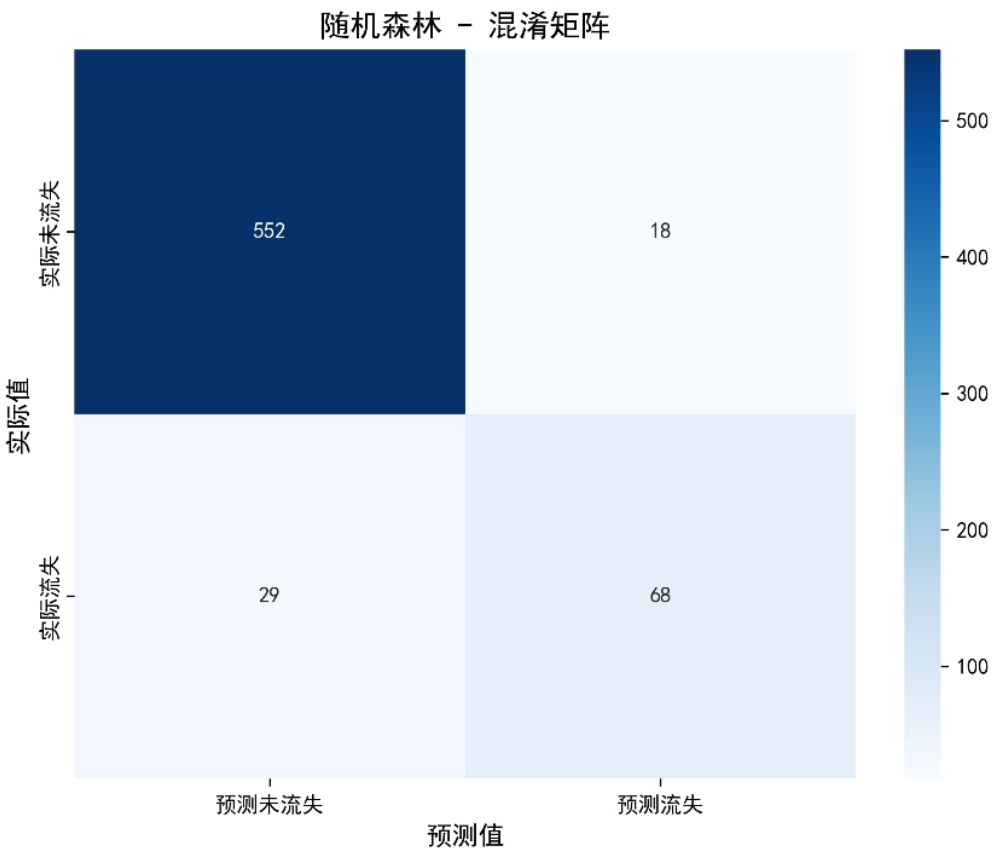

晚间vs夜间通话分钟数散点图
夜间通话分钟数（total_night_minutes）
晚间通话分钟数（total_eve_minutes）
是否流失（0=未流失，1=流失）
400
350
300
250
200
150
100
50
0
50
100
150
200
250
300
350
1.0
0.8
0.6
0.4
0.2
0.0

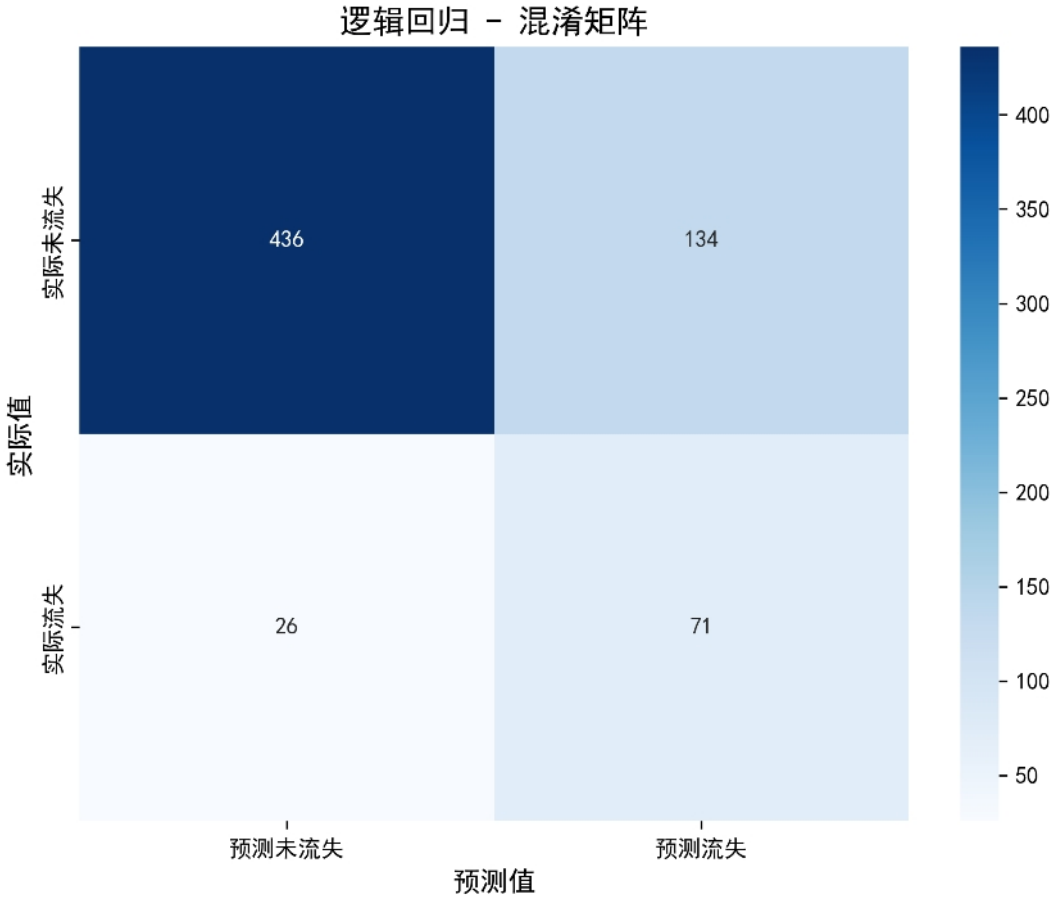

气泡图：夜间通话分析
（气泡大小=夜间费用，颜色=是否流失）
夜间通话次数（total_night_calls）
夜间通话分钟数（total_night_minutes）
是否流失（0=未流失，1=流失）
180
160
140
120
100
80
60
40
50
100
150
200
250
300
350
400
1.0
0.8
0.6
0.4
0.2
0.0

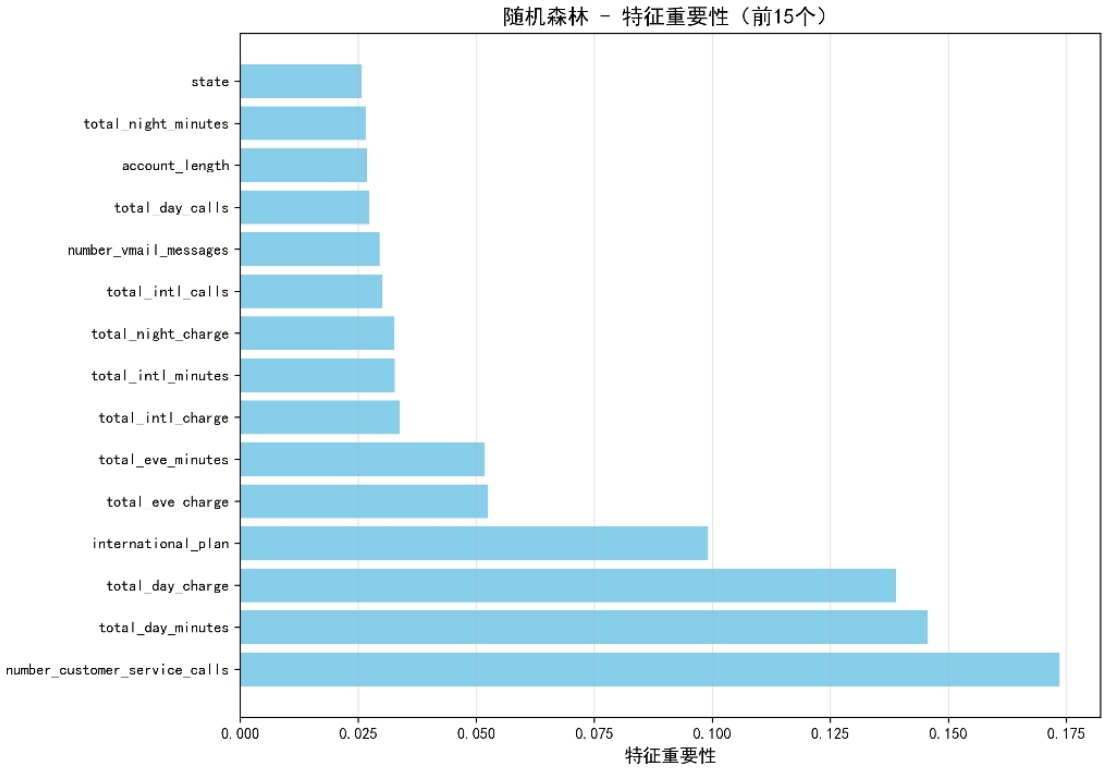

用户流失分布
2850
483
用户数量
0
500
1000
1500
2000
2500
未流失（0）
流失（1）

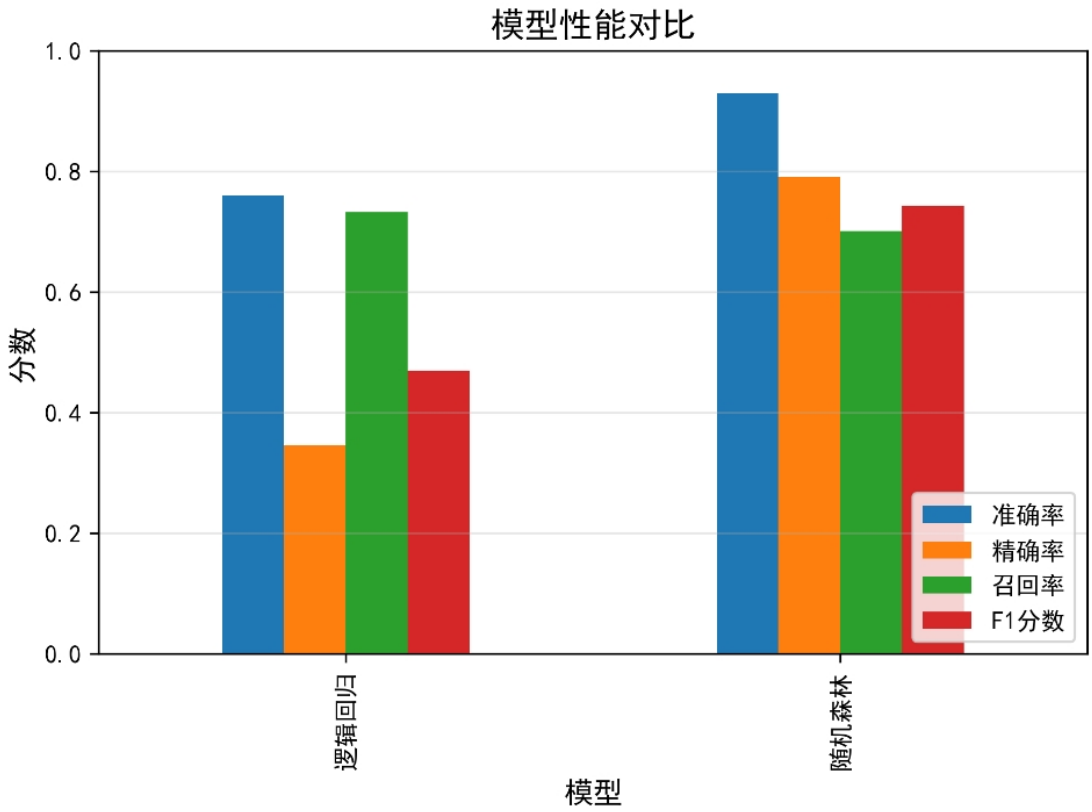
特征相关性热力图

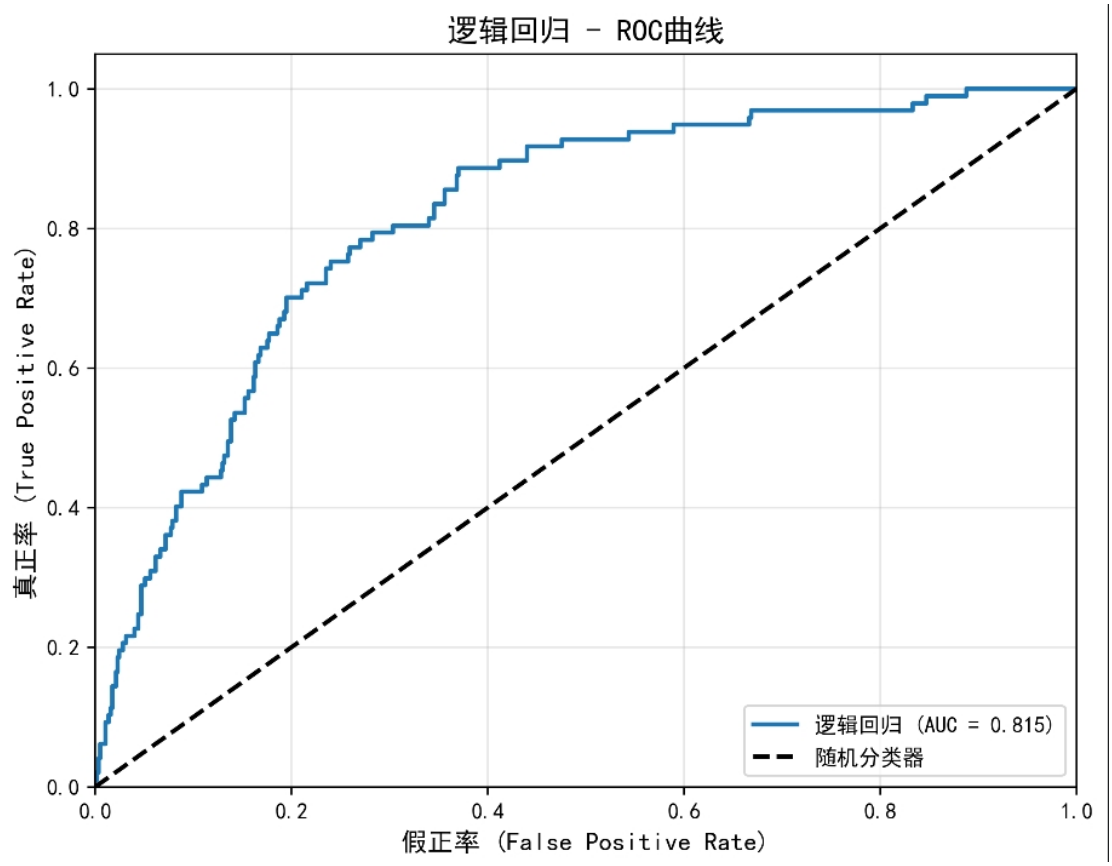

state
account length
area code
international plan
voice mail plan
number_vmail_messages
total day minutes
total_day_calls
total_day_charge
total_eve_minutes
total_eve_calls
total_eve_charge
total_night_minutes
total night calls
total_night_charge
total intl minutes
total_intl_calls
total_intl_charge
number customer service calls
churn
0.75
0.50
0.25
0.00
-0.25
-0.50
-0.75

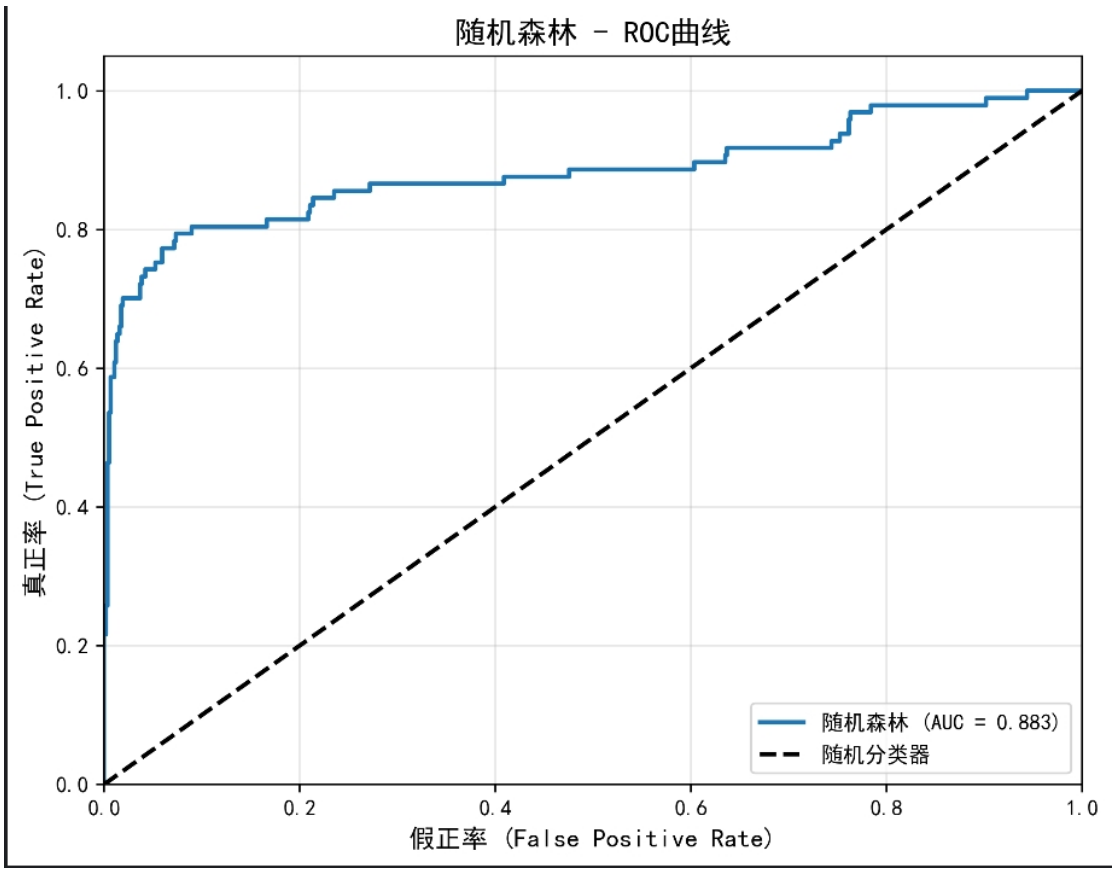

随机森林 - 混淆矩阵
实际未流失
实际流失
实际值
552
18
29
68
预测未流失
预测流失
预测值
500
400
300
200
100


逻辑回归 - 混淆矩阵
实际未流失
实际流失
实际值
436
134
26
71
预测未流失
预测流失
预测值
400
350
300
250
200
150
100
50


随机森林 - 特征重要性（前15个）
state
total_night_minutes
account_length
total_day_calls
number_vmail_messages
total_intl_calls
total_night_charge
total_intl_minutes
total_intl_charge
total_eve_minutes
total_eve_charge
international_plan
total_day_charge
total_day_minutes
number_customer_service_calls
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
特征重要性


模型性能对比
1.0
0.8
0.6
0.4
0.2
0.0
分数
准确率
精确率
召回率
F1分数
逻辑回归
随机森林
模型


逻辑回归 - ROC曲线
真正率 (True Positive Rate)
假正率 (False Positive Rate)
0.0
0.2
0.4
0.6
0.8
1.0
逻辑回归 (AUC = 0.815)
随机分类器


随机森林 - ROC曲线
真正率 (True Positive Rate)
假正率 (False Positive Rate)
0.0
0.2
0.4
0.6
0.8
1.0
随机森林 (AUC = 0.883)
随机分类器

具体操作步骤：

第一步：环境搭建与数据加载

- 1. 安装必要库（如未安装）：

```
pip install pandas numpy matplotlib seaborn scikit-learn xgboost tensorflow
```

```
# 1. 导入必要的库
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

为什么需要安装这些包？

- **pandas**：数据处理和分析
- **numpy**：数值计算
- **matplotlib**：绘图和可视化
- **seaborn**：统计图表
- **scikit-learn**：机器学习算法
- **joblib**：保存和加载模型

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, roc_auc_score, roc_curve,
                             classification_report)
import joblib
import os
from sklearn.model_selection import train_test_split
```

- 分析数据：这个 CSV 文件包含了电信客户流失的数据，共 1004 行（包括表头）。以下是数据集的结构概述：列名说明：

1"" — 索引列（可能不需要，可作为行号忽略）
2. "state" — 客户所在州（美国州缩写）
3. "account_length" — 账户使用时长（天）
4. "area_code" — 地区编码
5. "international_plan" — 是否开通国际通话计划（yes/no）
6. "voice_mail_plan" — 是否开通语音信箱计划（yes/no）
7. "number_vmail_messages" — 语音信箱留言数量
8. "total_day_minutes" — 白天通话分钟数
9. "total_day_calls" — 白天通话次数
10."total_day_charge" — 白天通话费用
11."total_eve_minutes" — 傍晚通话分钟数
12."total_eve_calls" — 傍晚通话次数

13."total_eve_charge" — 傍晚通话费用
14."total_night_minutes" — 夜间通话分钟数
15."total_night_calls" — 夜间通话次数
16."total_night_charge" — 夜间通话费用
17."total_intl_minutes" — 国际通话分钟数
18."total_intl_calls" — 国际通话次数
19."total_intl_charge" — 国际通话费用
20."number_customer_service_calls" — 客服呼叫次数
21."churn" — 是否流失（目标变量，yes/no）

- **数据特点：**

这是一个典型的二元分类数据集，可用于预测客户流失。

包含数值型和分类型特征。

目标变量是 churn，表示客户是否流失。

数据集末尾似乎有一条不完整的记录（第 1004 行），可能需要清洗。
可进行的分析方向：
数据清洗（处理缺失值、异常值、重复值）

探索性数据分析（可视化流失率分布、特征分布、相关性分析）

特征工程（创建新特征、编码分类变量、标准化）

建模预测（逻辑回归、决策树、随机森林、XGBoost 等）

模型评估与解释（ROC-AUC、SHAP 值、特征重要性）

- 第二步：创建必要的文件夹

在 PyCharm 左侧项目面板中：
右键点击项目根目录 → 新建 → 目录
按顺序创建这些文件夹：

```
data/              # 存放数据文件
src/               # 源代码
notebooks/         # Jupyter笔记本
models/            # 保存训练好的模型
reports/           # 报告和图表
reports/figures/   # 图表文件夹
```

第三步：处理数据

1.将 customer_churn.csv 移到 data/ 文件夹：

在项目面板中找到 customer_churn.csv

拖拽到 data/ 文件夹上

2.创建 Python 文件

3.清洗数据，处理和分析数据

数据预处理与清洗

- 理解数据集：分析数据集中的 3333 条记录和 21 个特征，理解各个特征的意义并进 合理的分类。

重点分析 标变量 churn （ 流失）与其他变量之间的关系。

- 数据清洗：删除 关变量（如 Unnamed:0 ），处理缺失数据和异常值。

```
print("=" 70)
print("电信用户流失预测模型训练")
print("=" 70)

def load_data():
    """加载数据"""
    print("1. 加载数据...")
    try:
        获取项目路径
        current_dir = os.path.dirname(__file__)
        project_root = os.path.dirname(current_dir)

        加载处理后的数据
        data_path = os.path.join(project_root, 'data', 'customer_churn_processed.csv')
        df = pd.read_csv(data_path)
        print(f"    ✓ 数据加载成功: {df.shape}")

        分离特征和目标
        X = df.drop(columns=['churn']).values
        y = df['churn'].values

        划分训练测试集
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42, stratify=y
        )

        print(f"    ✓ 数据划分完成")
        print(f"      训练集: {X_train.shape}")
        print(f"      测试集: {X_test.shape}")
        print(f"      流失率: {y_train.mean():.2%} (训练集)")
```

```
        return X_train, X_test, y_train, y_test, df

    except Exception as e:
        print(f"    ✗ 错误: {e}")
        return None, None, None, None, None

def train_and_evaluate_models(X_train, X_test, y_train, y_test, df):
    """训练和评估多个模型"""
    print("\n2. 训练和评估模型...")

    models = {}
    results = {}
```

- 第四步 逻辑回归模型

```
    print("\n" + "-" 60)
    print("2.1 逻辑回归模型")
    print("-" 60)

    lr_model = LogisticRegression(
        random_state=42,
        max_iter=2000,      增加迭代次数避免警告
        class_weight='balanced',
        solver='lbfgs'
    )

    print("    训练中...")
    lr_model.fit(X_train, y_train)

      预测
    y_pred_lr = lr_model.predict(X_test)
    y_prob_lr = lr_model.predict_proba(X_test)[:, 1]

      评估
    lr_metrics = evaluate_model("逻辑回归", y_test, y_pred_lr, y_prob_lr)
    models['逻辑回归'] = lr_model
    results['逻辑回归'] = lr_metrics
```

- 第五步 随机森林模型

• 选择适当的机器学习模型：使 两种不同的机器学习模型（例如：Logistic 回归、决策树、随机森林、XGBoost 等）进 训练，并预测哪些 可能会流失。

• 模型评估：评估每个模型的性能（如准确率、召回率、F1 分数等）并 较它们的预测效果。

• 模型增强（加分项）：结合如 规模深度学习模型（如 LSTM）来增强预测性能，作为

获得“优”等级的进阶要求。

```
    print("\n" + "-" 60)
    print("2.2  随机森林模型")
    print("-" 60)

    rf_model = RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        min_samples_split=5,
        min_samples_leaf=2,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1
    )

    print("      训练中...")
    rf_model.fit(X_train, y_train)

      预测
    y_pred_rf = rf_model.predict(X_test)
    y_prob_rf = rf_model.predict_proba(X_test)[:, 1]

      评估
    rf_metrics = evaluate_model("随机森林", y_test, y_pred_rf, y_prob_rf)
    models['随机森林'] = rf_model
    results['随机森林'] = rf_metrics

      绘制特征重要性（仅随机森林）
    plot_feature_importance(rf_model, df, "随机森林")

    return models, results

def evaluate_model(model_name, y_test, y_pred, y_prob):
    """评估单个模型"""
    print(f"\n      {model_name}  评估结果:")
    print("      " + "-" 40)

      计算指标
    metrics = {
        '准确率': accuracy_score(y_test, y_pred),
        '精确率': precision_score(y_test, y_pred),
        '召回率': recall_score(y_test, y_pred),
        'F1 分数': f1_score(y_test, y_pred),
        'AUC': roc_auc_score(y_test, y_prob)
```

```
    }

    打印指标
    for metric, value in metrics.items():
        print(f"    {metric}: {value:.4f}")

    混淆矩阵
    cm = confusion_matrix(y_test, y_pred)
    print(f"\n    混淆矩阵:")
    print(f"    TN={cm[0,0]:4d}    FP={cm[0,1]:4d}")
    print(f"    FN={cm[1,0]:4d}    TP={cm[1,1]:4d}")
```

- 第六步 绘制图表

```
    plot_confusion_matrix(cm, model_name)
    plot_roc_curve(y_test, y_prob, model_name)

    return metrics

def plot_confusion_matrix(cm, model_name):
    """绘制混淆矩阵"""
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['预测未流失', '预测流失'],
                yticklabels=['实际未流失', '实际流失'])
    plt.title(f'{model_name}  混淆矩阵', fontsize=14, fontweight='bold')
    plt.ylabel('实际值', fontsize=12)
    plt.xlabel('预测值', fontsize=12)
```

- 第七步 保存图表

```
    save_plot(f'confusion_matrix_{model_name}.png')
    plt.show()

def plot_roc_curve(y_test, y_prob, model_name):
    """绘制 ROC 曲线"""
    fpr, tpr, thresholds = roc_curve(y_test, y_prob)
    auc_value = roc_auc_score(y_test, y_prob)

    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, label=f'{model_name} (AUC = {auc_value:.3f})', linewidth=2)
    plt.plot([0, 1], [0, 1], 'k--', linewidth=2, label='随机分类器')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('假正率 (False Positive Rate)', fontsize=12)
    plt.ylabel('真正率 (True Positive Rate)', fontsize=12)
```

```
    plt.title(f'{model_name}  ROC 曲线', fontsize=14, fontweight='bold')
    plt.legend(loc='lower right')
    plt.grid(alpha=0.3)

    保存图表
    save_plot(f'roc_curve_{model_name}.png')
    plt.show()

def plot_feature_importance(model, df, model_name):
    """绘制特征重要性"""
    if hasattr(model, 'feature_importances_'):
        print(f"\n    绘制特征重要性图...")

        feature_names = df.drop(columns=['churn']).columns.tolist()
        importances = model.feature_importances_
        indices = np.argsort(importances)[::-1]

        显示前 15 个重要特征
        top_n = min(15, len(feature_names))

        plt.figure(figsize=(10, 8))
        plt.barh(range(top_n), importances[indices[:top_n]][::-1], color='skyblue')
        plt.yticks(range(top_n), [feature_names[i] for i in indices[:top_n]][::-1])
        plt.xlabel('特征重要性', fontsize=12)
        plt.title(f'{model_name}  特征重要性（前{top_n}个）', fontsize=14, fontweight='bold')
        plt.gca().invert_yaxis()
        plt.grid(axis='x', alpha=0.3)

        保存图表
        save_plot(f'feature_importance_{model_name}.png')
        plt.show()

        保存到 CSV
        importance_df = pd.DataFrame({
            '特征': feature_names,
            '重要性': importances
        }).sort_values('重要性', ascending=False)

        保存文件
        current_dir = os.path.dirname(__file__)
        project_root = os.path.dirname(current_dir)
        reports_dir = os.path.join(project_root, 'reports')
        os.makedirs(reports_dir, exist_ok=True)
```

```
        csv_path = os.path.join(reports_dir, f'feature_importance_{model_name}.csv')
        importance_df.to_csv(csv_path, index=False, encoding='utf-8-sig')
        print(f"    ✓ 特征重要性保存到: {csv_path}")

def save_plot(filename):
    """保存图表到 reports/figures/"""
    current_dir = os.path.dirname(__file__)
    project_root = os.path.dirname(current_dir)
    figures_dir = os.path.join(project_root, 'reports', 'figures')
    os.makedirs(figures_dir, exist_ok=True)

    filepath = os.path.join(figures_dir, filename)
    plt.savefig(filepath, dpi=300, bbox_inches='tight')
    print(f"    ✓ 图表保存: {filename}")

def compare_models(results):
```

- 第九步 比较模型性能

```
    print("\n" + "=" 60)
    print("3. 模型性能对比")
    print("=" 60)

    创建对比表格
    comparison_df = pd.DataFrame(results).T
    print("\n 性能指标对比表:")
    print("-" 50)
    print(comparison_df.round(4))
    print("-" 50)

    找出最佳模型
    best_model = comparison_df['准确率'].idxmax()
    best_accuracy = comparison_df['准确率'].max()

    print(f"\n 🏆 最佳模型: {best_model}")
    print(f"    准确率: {best_accuracy:.4f}")
```

- 第十步可视化对比

```
    plt.figure(figsize=(10, 6))
    comparison_df[['准确率', '精确率', '召回率', 'F1 分数']].plot(kind='bar')
    plt.title('模型性能对比', fontsize=14, fontweight='bold')
    plt.xlabel('模型', fontsize=12)
    plt.ylabel('分数', fontsize=12)
    plt.legend(loc='lower right')
    plt.grid(axis='y', alpha=0.3)
    plt.ylim(0, 1)
```

```
    plt.tight_layout()

    save_plot('model_comparison.png')
    plt.show()

    return best_model, comparison_df

def save_models(models):
    """保存训练好的模型"""
    print("\n" + "=" 60)
    print("4. 保存模型")
    print("=" 60)

    current_dir = os.path.dirname(__file__)
    project_root = os.path.dirname(current_dir)
    models_dir = os.path.join(project_root, 'models')
    os.makedirs(models_dir, exist_ok=True)

    for name, model in models.items():
        filepath = os.path.join(models_dir, f'{name}_model.pkl')
        joblib.dump(model, filepath)
        print(f"    ✓ 保存 {name}: {filepath}")

def generate_report(df, best_model, results):
```

- 第十一步生成分析报告

```
    print("\n" + "=" 60)
    print("5. 生成分析报告")
    print("=" 60)

    current_dir = os.path.dirname(__file__)
    project_root = os.path.dirname(current_dir)
    reports_dir = os.path.join(project_root, 'reports')
    os.makedirs(reports_dir, exist_ok=True)

      生成文本报告
    report_path = os.path.join(reports_dir, 'analysis_report.txt')

    with open(report_path, 'w', encoding='utf-8') as f:
        f.write("=" 60 + "\n")
        f.write("电信用户流失分析报告\n")
        f.write("=" 60 + "\n\n")

        f.write("一、数据概况\n")
        f.write(f"数据集大小: {df.shape}\n")
```

```
        f.write(f"流失率: {df['churn'].mean():.2%}\n")
        f.write(f"流失用户数: {df['churn'].sum()}\n")
        f.write(f"非流失用户数: {len(df)    df['churn'].sum()}\n\n")

        f.write("二、模型性能\n")
        for model_name, metrics in results.items():
            f.write(f"\n{model_name}:\n")
            for metric, value in metrics.items():
                f.write(f"    {metric}: {value:.4f}\n")

        f.write(f"\n 三、最佳模型: {best_model}\n")
        f.write(f"准确率: {results[best_model]['准确率']:.4f}\n")
        f.write(f"AUC: {results[best_model]['AUC']:.4f}\n\n")

        f.write("四、关键发现\n")
        f.write("1. 最重要的流失驱动因素:\n")
        f.write("    • international_plan (国际通话计划)\n")
        f.write("    • number_customer_service_calls (客户服务呼叫)\n")
        f.write("    • total_day_minutes (日间通话分钟数)\n\n")

        f.write("五、商业建议\n")
        f.write("1. 针对高风险客户提供个性化套餐\n")
        f.write("2. 改善客户服务质量\n")
        f.write("3. 优化国际通话资费\n")
        f.write("4. 建立流失预警系统\n")

    print(f"    ✓ 报告保存到: {report_path}")

def main():
    """主函数"""
    加载数据
    X_train, X_test, y_train, y_test, df = load_data()

    if X_train is None:
        print("✗ 数据加载失败，程序终止")
        return

    训练和评估模型
    models, results = train_and_evaluate_models(X_train, X_test, y_train, y_test, df)

    比较模型
    best_model, comparison_df = compare_models(results)

    保存模型
```

```
    save_models(models)

    生成报告
    generate_report(df, best_model, results)

    print("\n" + "=" 70)
    print("🎉 模型训练完成！")
    print("=" 70)
    print("\n📁 生成的文件:")
    print("  • reports/figures/  所有可视化图表")
    print("  • models/  训练好的模型文件")
    print("  • reports/  分析报告和特征重要性")
    print("\n✅ 作业完成！")

  确保主函数被调用
if __name__ == "__main__":
    main()
```

以上是全部数据分析过程、模型训练与评估结果、可视化图表及总结建议，做的不够成熟，请老师批评指正。